

Explain grid method with example

The first manager, Pack is very good for simple applications. But to arrange the widgets in the tabular format we have to use grid manager. It provides the option for rows and columns. Grid is easier to learn and to use and produces nicer layouts. It places the widgets in a 2-dimensional table, which consists of a number of rows and columns. The position of a widget in a GUI is defined by a row and a column number. The size of the grid we don't have to define because the manager automatically determines the best dimensions for the widgets used.

The Grid Layout Manager Example with 3 Rows and 3 Columns:

Row 0; Col 0 **Row 0; Col 1** **Row 0; Col 2**

Row 1; Col 0 Row 1; Col 1 Row 1; Col 2

Row 2; Col 0 Row 2; Col 1 Row 2; Col 2

The grid method options:

- **row:** It is used to specify the number of the row where we want to place the widget.
- **column:** It is used to specify the number of the column where we want to place the widget.
- **columnspan:** It is used to specify the number of columns that must be occupied by the widget.
- **rowspan:** It is used to specify the number of rows that must be occupied by the widget.

Example

```
from tkinter import *
root = Tk()
root.title("User Form")
Label(root, text="Name:").grid(row=0, column=0)
Entry(root).grid(row=0, column=1)
Label(root, text="Mobile:").grid(row=1, column=0)
Entry(root).grid(row=1, column=1)

Label(root, text="Age:").grid(row=2, column=0)
Entry(root).grid(row=2, column=1)
Button(root, text="Submit").grid(row=3, column=0)
Button(root, text="Exit", command=root.quit).grid(row=3, column=1)
root.mainloop()
```

Explain pack method with example

It is the easiest to use among the three geometry managers of Tk and Tkinter. But this layout manager has limited options compared to the grid and place managers.

The pack method options:

- **Expand:** It expands the widget to use all the remaining space after the other widgets have been informed.
- **resized.fill:** It defines how the widget should fill up the space provided by its parcel. Possible values: x, y, both, and none.
- **ipadx, ipady:** They are used along with the fill option to define the space in pixels around the widget.
 - ipadx - Internal padding, horizontally.
 - ipady - Internal padding, vertically.
- **padx, pady:** They define the space in pixels between widgets.
 - padx - External padding, horizontally.
 - pady - External padding, vertically.
- **side:** It defines the side where we want to place the widget. Possible values: top, bottom, left, and right.

Example:

```
from tkinter import *  
root = Tk()  
Button(root, text="Red", bg="red", fg="white").pack()  
Button(root, text="Green", bg="green", fg="white").pack()  
Button(root, text="Blue", bg="blue", fg="white").pack()  
Button(root, text="Orange", bg="orange", fg="white").pack()  
mainloop()
```

Explain layout manager in detail

To arrange the widgets in the GUI we have a layout managers in python. Tkinter provides three layout managers or geometry manager:

- pack
- grid
- place

The three layout managers should never be mixed in the same master window. In one window use only one of the layout manager. Layout managers serve various functions which are as follows:

- They arrange widgets on the window. It includes determining the size and position of components. Widgets can provide size and alignment information to the geometry managers, but the geometry managers have always the final control on the positioning and sizing.
- They register widgets with the underlying windowing system.
- They manage the display of widgets on the screen.
- **pack** → Arranges widgets in a block-like structure (top, bottom, left, right). Best for simple layouts.
- **grid** → Organizes widgets in a table (rows & columns). Great for structured layouts.
- **place** → Positions widgets at exact (x, y) coordinates. Used for precise control.

Explain radio button with example

A **Radio Button** in Tkinter allows the user to select **one option** from multiple choices. It is commonly used in forms where only one selection is allowed at a time (e.g., selecting gender, selecting a payment method).

Example

```
import tkinter as tk

root = tk.Tk()

root.title("Radio Button Example")

choice = tk.StringVar(value="Option 1")

tk.Radiobutton(root, text="Option 1", variable=choice, value="Option 1").pack()

tk.Radiobutton(root, text="Option 2", variable=choice, value="Option 2").pack()

root.mainloop()
```

Explain radio button and write a program to display of programming language with radio button

A **Radio Button** is a GUI component that allows the user to select **one option** from multiple choices. In Tkinter, radio buttons are created using the Radiobutton widget and are linked using a StringVar or IntVar so that only one option can be selected at a time.

```

import tkinter as tk

root = tk.Tk()
root.title("Select a Programming Language")

selected_lang = tk.StringVar(value="None")

def show_selection():
    label.config(text="Selected: " + selected_lang.get())

languages = ["Python", "Perl", "Java", "C++", "C", "C#"]
for lang in languages:
    tk.Radiobutton(root, text=lang, variable=selected_lang, value=lang,
command=show_selection).pack()
label = tk.Label(root, text="Selected: None")
label.pack()
root.mainloop()

```

Explain check button with example

A **Check Button** (or **Checkbutton**) in Python is a GUI component that allows users to select or deselect options. It is commonly used in Tkinter (Python's standard GUI library). A Checkbutton can be used for selecting multiple options independently.

Example

#CheckButton In Python

```

from tkinter import *
t = Tk()
CheckVar1 = IntVar()
CheckVar2 = IntVar()
C1 = Checkbutton(t, text = "Music", variable = CheckVar1, \
onvalue = 1, offvalue = 0, height=5, \
width = 20)
C2 = Checkbutton(t, text = "Video", variable = CheckVar2, \
onvalue = 1, offvalue = 0, height=5, \
width = 20)
C1.pack()
C2.pack()
t.mainloop()

```

write a program to display a message with some color and font effect

```

from tkinter import *

root = Tk()

W = Message(root, text="Hello everyone", width=500)

```

```
W.config(bg='lightgreen', font=('Times', 24, 'italic', 'bold'))
```

```
W.pack() # Pack the widget to display it
```

```
root.mainloop()
```

Write a program to display canvas with line, oval and arc

```
from tkinter import *
```

```
root = Tk()
```

```
canvas = Canvas(root, width=300, height=200)
```

```
canvas.pack()
```

```
canvas.create_line(50, 50, 250, 50, fill="blue", width=3)
```

```
canvas.create_oval(50, 80, 150, 180, outline="red", width=3)
```

```
canvas.create_arc(150, 80, 250, 180, start=0, extent=180, fill="green")
```

```
root.mainloop()
```

Design a simple database application using python that stores the record of students Student table(student ID, student name, student address, course name)

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
import mysql.connector as mysql
```

```
def put():
```

```
    conn = mysql.connect(  
        user="pallavi",  
        password="abc@1234",  
        host="localhost",  
        database="college_db"
```

```
    )
```

```
    cur = conn.cursor()
```

```
    cur.execute("INSERT INTO student (student_name, student_address, course_name)  
VALUES (%s, %s, %s)",
```

```
        (nameEntry.get(), addressEntry.get(), courseEntry.get()))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    messagebox.showinfo("Success", "Student record inserted!")
```

```
root = Tk()
```

```

root.title("Insert Student Record")

Label(root, text="Student Name:").grid(row=0, column=0)
Label(root, text="Address:").grid(row=1, column=0)
Label(root, text="Course Name:").grid(row=2, column=0)
nameEntry = Entry(root, width=25)
nameEntry.grid(row=0, column=1)
addressEntry = Entry(root, width=25)
addressEntry.grid(row=1, column=1)
courseEntry = Entry(root, width=25)
courseEntry.grid(row=2, column=1)
Button(root, text="Insert", command=put).grid(row=3, column=1)
root.mainloop()

```

Design a simple database application using python GUI that stores the records of patients and retrieve the same
Patient table (patient id, patient name, age , address)

```

from tkinter import *
from tkinter import messagebox
import mysql.connector as mysql

# Database Connection
conn = mysql.connect(user="pallavi", password="abc@1234", host="localhost",
database="hospital_db")
cur = conn.cursor()

# Insert Patient Data
def insert():
    cur.execute("INSERT INTO patient (patient_id, patient_name, age, address)
VALUES (%s, %s, %s, %s)",
                (idEntry.get(), nameEntry.get(), ageEntry.get(), addressEntry.get()))
    conn.commit()
    messagebox.showinfo("Success", "Record Inserted!")

# Retrieve Patient Data
def retrieve():
    cur.execute("SELECT * FROM patient")
    displayBox.delete(1.0, END)
    for row in cur.fetchall():
        displayBox.insert(END, f'{row}\n')

# GUI Design
root = Tk()
root.title("Patient Records")

```

```

Label(root, text="Patient ID:").grid(row=0, column=0)
Label(root, text="Name:").grid(row=1, column=0)
Label(root, text="Age:").grid(row=2, column=0)
Label(root, text="Address:").grid(row=3, column=0)

idEntry, nameEntry, ageEntry, addressEntry = Entry(root), Entry(root), Entry(root),
Entry(root)
idEntry.grid(row=0, column=1)
nameEntry.grid(row=1, column=1)
ageEntry.grid(row=2, column=1)
addressEntry.grid(row=3, column=1)

Button(root, text="Insert", command=insert).grid(row=4, column=1)
Button(root, text="Retrieve", command=retrieve).grid(row=5, column=1)

displayBox = Text(root, width=40, height=5)
displayBox.grid(row=6, column=0, columnspan=2)

root.mainloop()
conn.close()close()

```

Design a database application using python GUI to modify the specific record of product using product id from the database and display modify record
Product table(product id, product name , quantity)

```

from tkinter import *
from tkinter import messagebox
import mysql.connector as mysql

# Database Connection
conn = mysql.connect(user="pallavi", password="abc@1234", host="localhost",
database="store_db")
cur = conn.cursor()

# Update Product Data
def update():
    cur.execute("UPDATE product SET product_name=%s, quantity=%s WHERE
product_id=%s",
                (nameEntry.get(), quantityEntry.get(), idEntry.get()))
    conn.commit()
    messagebox.showinfo("Success", "Record Updated!")

# Retrieve Updated Record
def retrieve():
    cur.execute("SELECT * FROM product WHERE product_id=%s", (idEntry.get(),))
    displayBox.delete(1.0, END)

```

```

row = cur.fetchone()
if row:
    displayBox.insert(END, f'{row}\n')
else:
    displayBox.insert(END, "No Record Found")

# GUI Design
root = Tk()
root.title("Modify Product")

Label(root, text="Product ID:").grid(row=0, column=0)
Label(root, text="Product Name:").grid(row=1, column=0)
Label(root, text="Quantity:").grid(row=2, column=0)

idEntry, nameEntry, quantityEntry = Entry(root), Entry(root), Entry(root)
idEntry.grid(row=0, column=1)
nameEntry.grid(row=1, column=1)
quantityEntry.grid(row=2, column=1)

Button(root, text="Update", command=update).grid(row=3, column=1)
Button(root, text="Retrieve", command=retrieve).grid(row=4, column=1)

displayBox = Text(root, width=40, height=3)
displayBox.grid(row=5, column=0, columnspan=2)
root.mainloop()
conn.close()

Explain update Command

Modifies existing records in a table.
UPDATE table_name SET column1 = value1 WHERE condition;
Example
import mysql.connector

conn = mysql.connector.connect(user="pallavi", password="abc@1234",
host="localhost", database="store_db")
cur = conn.cursor()

cur.execute("UPDATE product SET quantity = 20 WHERE product_id = 1")
conn.commit()

print("Product updated successfully!")

conn.close()

```

Explain insert command

Adds new records to a database table.

Syntax: INSERT INTO table_name (col1, col2) VALUES (val1, val2);

```
import mysql.connector
conn = mysql.connector.connect(user="pallavi", password="abc@1234",
host="localhost", database="store_db")
cur = conn.cursor()

cur.execute("INSERT INTO product (product_id, product_name, quantity) VALUES
(1, 'Laptop', 10)")
conn.commit()

print("Product inserted successfully!")

conn.close()
```

UNIT-3

2. Advantage of Django

Rapid Development – Django follows the "batteries-included" approach, providing built-in features like authentication, admin panel, and ORM, reducing development time.

Scalability – Django is highly scalable and can handle high traffic loads, making it suitable for large applications.

Security – Django includes built-in security features such as protection against SQL injection, XSS, CSRF, and clickjacking.

Powerful ORM (Object-Relational Mapper) – The Django ORM simplifies database interactions and supports multiple databases like PostgreSQL, MySQL, and SQLite.

Automatic Admin Interface – Django automatically generates an admin panel for managing application data.

SEO-Friendly – Django allows the creation of human-readable URLs, making it easier for search engines to index web pages.

Versatility – Django is used for web applications, data science platforms, AI models, and even IoT applications.

Excellent Community Support – Django has a large community that provides extensive documentation, tutorials, and third-party packages.

Built-in Caching – Django supports caching mechanisms to enhance performance and speed up applications.

REST Framework Support – Django REST Framework (DRF) makes it easy to build robust REST APIs for web and mobile applications.

Explain url.py and view.py in Django

1. Role of urls.py:

It acts as a router that directs incoming HTTP requests to the appropriate view function.

It organizes and manages URL patterns for better readability and maintainability.

It supports path parameters to dynamically pass values to views.

Example

```
from django.contrib import admin
from django.urls import path
from home.views import *
```

```
urlpatterns = [
    path("", home, name="home"),

    path('success-page/', success_page, name="success_page"),
    path('admin/', admin.site.urls),
]
```

2. Role of views.py:

It processes requests and returns a response (HTML, JSON, or another format).

It interacts with models to fetch or update data in the database.

It uses Django's template engine to render dynamic HTML pages.

Example

```
from django.shortcuts import render
from django.http import HttpResponse
```

```
def home(request):  
    return HttpResponse('<h1>Hello, World!</h1>')
```

```
def success_page(request):  
    return HttpResponse('<h1>Success!</h1>')
```

What is a Django template and how does it work?

Answer: Django templates are used to generate HTML dynamically. They allow you to embed Python-like code in HTML files. The template engine processes these templates and returns the rendered HTML as a response to the client.

Example:

```
<html>  
    <body>  
        Hello World!!!<p>Today is {{today}}</p>  
    </body>  
</html>
```

Difference between models.py and views.py in Django

models.py:

Defines the structure of the database using Django's ORM (Object-Relational Mapping).

Contains Python classes that represent database tables, with attributes corresponding to fields.

Provides built-in methods to interact with the database (e.g., save(), delete(), filter(), all()).

Allows defining relationships between tables using ForeignKey, OneToOneField, and ManyToManyField.

Example

```
from django.db import models
```

```
class Member(models.Model):
```

```
firstname = models.CharField(max_length=255)
```

```
lastname = models.CharField(max_length=255)
```

views.py:

Handles the logic of processing user requests and returning responses.

Interacts with models.py to fetch, update, or delete data from the database.

Can return responses in different formats such as HTML, JSON, or XML.

Uses Django's built-in functions like render(), redirect(), and HttpResponseRedirect().

Can be function-based views (FBV) or class-based views (CBV).

Example

```
from django.shortcuts import render
```

```
from django.http import HttpResponseRedirect
```

```
def home(request):
```

```
    return HttpResponseRedirect('<h1>Hello, World!</h1>')
```

Design Django application to print Hello World

Step 1: Install Virtual Environment

```
pip install virtualenv
```

Installs virtualenv, a tool to create isolated Python environments.

Step 2: Create and Activate a Virtual Environment

```
virtualenv env
```

Creates a virtual environment named 'env'.

```
cd env/Scripts
```

```
activate
```

Navigates to Scripts folder and activates the virtual environment.

Step 3: Install Django

```
pip install Django
```

Installs Django, a web framework for Python.

Step 4: Start a Django Project

`django-admin startproject core`
Creates a new Django project named 'core'.

Step 5: Navigate to the Project Directory

`cd core`
Moves into the project directory.

Step 6: Open Project in VS Code

`code .`
Opens the project in Visual Studio Code.

Step 7: Create a Django App

`python manage.py startapp home`
Creates an app named 'home'.

Step 8: Run the Django Server

`python manage.py runserver`
Starts the local Django development server at `http://127.0.0.1:8000/`.

Step 9: Migrate Database Changes

`python manage.py migrate`
Applies pending database migrations.

Step 10: Create a Simple View (home)

In `views.py`:
`from django.shortcuts import render`
`from django.http import HttpResponse`

```
def home(request):  
    return HttpResponse('<h1>Hello, World!</h1>')
```

Defines a function 'home' that returns 'Hello, World!' when accessed.

Step 11: Configure URL Routing

In `urls.py`:
`from django.contrib import admin`
`from django.urls import path`
`from home.views import *`

```
urlpatterns = [  
    path("", home, name='home'),  
    path('admin/', admin.site.urls),  
]
```

Maps URLs to views.

Step 12: Running the Server Again

`python manage.py runserver`
Starts the development server.

Final Output

When you visit `http://127.0.0.1:8000/`, you will see:

Hello, World!

Create Django Application to print All the Best

Step 1: Install Virtual Environment

`pip install virtualenv`
Installs virtualenv, a tool to create isolated Python environments.

Step 2: Create and Activate a Virtual Environment

`virtualenv env`
Creates a virtual environment named 'env'.

`cd env/Scripts`
`activate`
Navigates to Scripts folder and activates the virtual environment.

Step 3: Install Django

`pip install Django`
Installs Django, a web framework for Python.

Step 4: Start a Django Project

`django-admin startproject core`
Creates a new Django project named 'core'.

Step 5: Navigate to the Project Directory

`cd core`
Moves into the project directory.

Step 6: Open Project in VS Code

`code .`
Opens the project in Visual Studio Code.

Step 7: Create a Django App

```
python manage.py startapp home
```

Creates an app named 'home'.

Step 8: Run the Django Server

```
python manage.py runserver
```

Starts the local Django development server at <http://127.0.0.1:8000/>.

Step 9: Migrate Database Changes

```
python manage.py migrate
```

Applies pending database migrations.

Step 10: Create a Simple View (home)

In `views.py`:

```
from django.shortcuts import render
from django.http import HttpResponse
```

```
def home(request):
    return HttpResponse('<h1>All the Best</h1>')
```

Defines a function 'home' that returns 'All the Best' when accessed.

Step 11: Configure URL Routing

In `urls.py`:

```
from django.contrib import admin
from django.urls import path
from home.views import *
```

```
urlpatterns = [
    path("", home, name='home'),
    path('admin/', admin.site.urls),
]
```

Maps URLs to views.

Step 12: Running the Server Again

```
python manage.py runserver
```

Starts the development server.

Final Output

When you visit <http://127.0.0.1:8000/>, you will see:

All the Best

